Du C au C++

1 · du C au C++

```
#include <iostream>          // sans .h
bool ok = true;              // type natif
int *p = nullptr;            // remplace NULL
auto x = 3.0;                // type déduit : double
for (int v : tab) { ... }    // parcours de conteneur
for (auto &v : tab) v *= 2;  // par réf. : modifie
int f(int a, int b = 0);     // argument par défaut
```

Surcharge de fonctions

```
void print(int v);           // même nom,
void print(double v);        // paramètres différents
void print(const char *s);
int  print(int v);           // ERREUR : retour seul
```

Signature = nom + portée + paramètres (+ `const`), **pas le retour** ; encodée dans le symbole : `print(int)` → `_Z5printi`. `extern "C"` pour lier du C.

Références

```
int i = 42;
int &r = i;                  // alias, à initialiser
r = 10;                      // i vaut 10 (pas de *)
void inc(int &x) { x++; }    // paramètre de sortie
void show(const Big &b);     // entrée sans copie
inc(i);                      // appel sans &
```

	`T *p` pointeur	`T &r` référence
Init.	facultative	**obligatoire**
Nul	`nullptr`	impossible
Réaff.	`p = &autre;`	non : `r = b` copie b
Arithm.	`p++` `p + i`	non
Accès	`*p` `p->m`	`r` `r.m`

⚠ **Piège :** `f(int)` et `f(int &)` ensemble : `f(i)` est ambigu. Ne jamais renvoyer une référence vers une variable locale.

Les 4 casts

`static_cast<T>(x)`	vérifié à la compilation
`dynamic_cast<D *>(b)`	à l'exécution ; sinon `nullptr`
`const_cast<T *>(p)`	ajoute / retire `const`
`reinterpret_cast<T>(x)`	relit les bits bruts
`(T)x`	cast C : à éviter

```
Base *b = new Derived();
if (Derived *d = dynamic_cast<Derived *>(b))
    d->execute();            // b est bien un Derived
auto *r = reinterpret_cast<Regs *>(0x40001000);
```

`dynamic_cast` : classe polymorphe (une méthode `virtual`) exigée.

enum class et namespace

```
enum class Color : uint8_t { Red, Green, Blue };
Color c = Color::Red;        // portée obligatoire
int i = static_cast<int>(c); // conversion explicite
namespace phelma::proj1 { void print(); }  // C++17
phelma::proj1::print();
namespace pp = phelma::proj1; // alias
using namespace std;         // jamais dans un .hpp
```

Anatomie d'une classe

2 · classes

```
// Circle.hpp : interface
#pragma once                 // ou garde #ifndef
class Circle {
public:
    Circle(double r = 1.0);  // constructeur
    ~Circle();               // destructeur
    double area() const;     // ne modifie pas *this
    static int count();      // sans objet, sans this
private:
    double radius;           // attribut
    static int nb;           // un seul pour tous
};
// Circle.cpp : implémentation
#include "Circle.hpp"
int Circle::nb = 0;          // définition du static
Circle::Circle(double r) : radius(r) { nb++; }
Circle::~Circle() { nb--; }
double Circle::area() const {
    return 3.14159 * radius * radius;
}
// main.cpp
Circle c(2.0);               // pile : auto-détruit
Circle *pc = new Circle(3);  // tas
pc->area();
delete pc;                   // destr. + libération
Circle::count();
```

Accès vérifié à la compilation seulement. Un objet ne contient que ses attributs ; les méthodes reçoivent `this` en paramètre caché.

Construction et destruction

```
class Duration {
public:
    Duration();                          // par défaut
    explicit Duration(long s);           // pas de conversion
    Duration(const Duration &o);         // copie
    Duration &operator=(const Duration &o);
    ~Duration();
private:
    long value;                          // pas const (operator=)
    std::string unit;
};
Duration::Duration(long s)
    : value(s), unit("s") {}             // liste d'init.
Duration d1(60);                         // constructeur
Duration d2 = d1;                        // constr. de copie
d2 = d1;                                 // operator=
void f(Duration d);                      // copiée à l'appel
Duration d3 = 60;                        // ERREUR : explicit
```

Liste init.	obligatoire pour : membres `const`, références, membres sans constructeur par défaut, classe de base à paramètres
Ordre	celui de **déclaration** des membres, pas celui de la liste
Cycle	base → membres → corps ; destruction en ordre inverse
Implicites	copie, `operator=`, destructeur ; `T()` seulement si aucun constructeur. + déplacement (C++11). `= default` les demande, `= delete` les interdit

Héritage et polymorphisme

3 · héritage

```
class Shape {                            // abstraite
public:
    Shape(std::string c) : color(c) {}
    virtual ~Shape() = default;          // OBLIGATOIRE
    virtual void draw() const = 0;       // virtuelle pure
    virtual double area() const { return 0; }
protected:
    std::string color;                   // pour les filles
};
class Rect : public Shape {
public:
    Rect(int w, int h) : Shape("red"), w(w), h(h) {}
    void draw() const override;          // redéfinition
    double area() const override { return w * h; }
private:
    int w, h;
};
void render(const Shape &s) {            // upcast
    s.draw();                            // -> Rect::draw()
}
std::vector<Shape *> v = {new Rect(1, 2)};
for (Shape *s : v) s->draw();            // appel dynamique
```

`virtual`	résolu à l'exécution, selon le type réel
`= 0`	virtuelle pure ⇒ classe abstraite, non instanciable ; que des virtuelles pures = interface
`override`	le compilateur vérifie la redéfinition · `final` l'interdit

⚠ **Piège :** sans destructeur `virtual`, `delete` d'un `Shape *` oublie la partie `Rect`. passer une base **par valeur** (`void f(Base b)`) tronque l'objet dérivé (*slicing*) : passer `const Base &`.

vtable et coût

```
struct A { uint32_t id, flags; void f(); };
struct B { uint32_t id, flags; virtual void f(); };
sizeof(A);                               // 8
sizeof(B);                               // 16 : + vptr (8 octets)
```

Une **vtable** (adresses des méthodes virtuelles) par classe, un **vptr** par objet. Appel virtuel = une indirection.

Accès et héritage

Accès	`public`	tout le monde
	`protected`	la classe et ses dérivées
	`private`	la classe (et ses `friend`)
Défaut	`struct`	tout `public`
	`class`	tout `private` ; héritage `private` par défaut
Composition	« possède un » : attribut (`Movie` a une `Duration`)	
Héritage	« est un » : `Rect` est une `Shape` (substituable)	
Mode	`: public B`	droits inchangés
	`: protected B`	public devient protected
	`: private B`	tout devient private (défaut)

```
class Product { public: float price; };
class Paper : public virtual Product { ... };
class Elec  : public virtual Product { ... };
class Smartpad : public Paper, public Elec {};
pad.price = 9.5f;                        // une seule copie de price
```

Sans `virtual` : deux `Product` (diamant), il faut écrire `pad.Paper::price`. Usage typique : hériter de plusieurs interfaces.

</> Surcharge d'opérateurs

4 · opérateurs · RAII

```
class Complex {
public:
    Complex(double re = 0, double im = 0)
        : re(re), im(im) {}
    Complex operator+(const Complex &o) const {
        return Complex(re + o.re, im + o.im);
    }
    Complex &operator+=(const Complex &o) {
        re += o.re;  im += o.im;
        return *this;                  // chaînage
    }
    Complex &operator++();             // ++c
    Complex operator++(int);           // c++ (int fictif)
    friend std::ostream &operator<<(std::ostream &os,
                                    const Complex &c);
private:
    double re, im;
};
std::ostream &operator<<(std::ostream &os,
                         const Complex &c) {
    return os << c.re << " + " << c.im << "i";
}
Complex a(1, 2), b = a + 3.0;          // 3.0 -> Complex
std::cout << a << '\n';
```

Méthode si l'opérande gauche est l'objet, sinon **fonction libre** (`cout << c`, `2.0 + c`). Interdits : `:: . .* ?: sizeof`.

Allocation et ressources

```
T *p = new T(args);                    // + constructeur
delete p;                              // + destructeur
T *t = new T[n];                       // n fois T()
delete[] t;                            // pas delete seul
auto u = std::make_unique<T>(args);    // delete auto
auto s = std::make_shared<T>(args);    // partagé
```

Échec : exception `std::bad_alloc`. Pointeurs intelligents : `<memory>`.

Règle des trois

```
class Buf {                            // possède un new[]
    int *data;
    size_t n;
public:
    Buf(size_t n) : data(new int[n]), n(n) {}
    ~Buf() { delete[] data; }          // 1. destructeur
    Buf(const Buf &o)                  // 2. copie profonde
        : data(new int[o.n]), n(o.n) {
        std::copy(o.data, o.data + n, data);
    }
    Buf &operator=(const Buf &o) {     // 3. affectation
        if (this != &o) {              // auto-affectation
            delete[] data;
            n = o.n;
            data = new int[n];
            std::copy(o.data, o.data + n, data);
        }
        return *this;
    }
};
```

Copie par défaut = superficielle : deux objets, un seul tableau ⇒ double `delete[]`. **RAII** : ressource prise dans le constructeur, rendue dans le destructeur.

Templates

5 · généricité · lambdas

```
template <typename T>
T maxOf(T a, T b) { return a > b ? a : b; }
maxOf(3, 7);                   // T = int, déduit
maxOf<double>(3, 7.5);         // T imposé

template <typename T1, typename T2>
class Pair {
public:
    Pair(T1 a, T2 b) : first(a), second(b) {}
    void print() const;
    T1 first;
    T2 second;
};
template <typename T1, typename T2>  // hors classe :
void Pair<T1, T2>::print() const {   //   tout répéter
    std::cout << first << " : " << second << '\n';
}
Pair<int, std::string> p(1, "un");

template <typename T, int N>  // paramètre valeur
class Array { T data[N]; };
template <>                   // spécialisation
class Array<bool, 8> { uint8_t bits; };
// type qui dépend d'un paramètre : typename devant
typename std::vector<T>::iterator it;
```

⚠ **Piège :** tout le code d'un template va dans le `.hpp` (instancié à la compilation), sinon `undefined reference` à l'édition de liens.

Fonctions comme objets

```
struct Greater {               // foncteur
    int thr;
    bool operator()(int v) const { return v > thr; }
};
Greater g{42};  g(50);         // true

int thr = 42;
auto f = [thr](int v) { return v > thr; };
auto h = [&](int v) -> bool { return v > thr; };
std::function<bool(int)> fn = f;   // <functional>
```

`[]`	ne capture rien
`[=]` `[&]`	tout par copie · tout par référence
`[x, &y]`	x copié, y par référence
`[this]`	accès aux membres de l'objet

Lambda = foncteur anonyme (intégrable, contrairement à un pointeur de fonction). `[&]` : danger si elle survit au bloc.

Algorithmes

```
std::sort(v.begin(), v.end());         // <algorithm>
std::sort(v.begin(), v.end(),
          [](int a, int b) { return a > b; });
auto it = std::find(v.begin(), v.end(), 3);
int n = std::count_if(v.begin(), v.end(), pred);
v.erase(std::remove_if(v.begin(), v.end(), pred),
        v.end());                      // supprime si pred
```

Flux et chaînes

6 · bibliothèque · move

```
std::cout << "x = " << x << '\n';   // sans flush
std::cerr << "erreur" << std::endl; // '\n' + flush
std::cin >> a >> b;                 // mot par mot
std::getline(std::cin, ligne);      // ligne entière
std::istringstream(ligne) >> mot >> n;
std::ifstream in("f.txt");          // ofstream : écrire
while (std::getline(in, ligne)) { ... }
std::cout << std::setw(8) << std::setprecision(3) << x;
```

std::string

`s.size()`	`s.empty()`	`s[i]` `s.at(i)`
`s + t`	`s += t`	`s == t` `s < t`
`s.find(t)`	`s.c_str()`	`s.substr(pos, n)`
`std::stoi(s)`	`std::stod(s)`	`std::to_string(x)`

`find` : `std::string::npos` si absent · `c_str()` : `const char *`

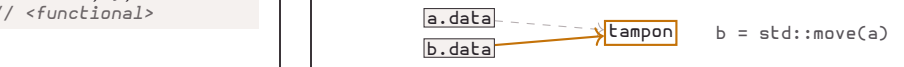
Conteneurs

```
std::vector<int> v = {3, 1, 2};
v.push_back(4);  v.pop_back();  v.size();
v[i];  v.at(i);                 // at : out_of_range
v.insert(v.begin() + 1, 9);  v.erase(v.begin());
std::map<std::string, int> m;
m["bob"] = 3;                   // crée la clé si absente
if (m.count("ann")) { ... }     // ou m.find(k) != m.end()
for (auto &[k, val] : m) ...    // C++17, trié par clé
for (auto it = v.begin(); it != v.end(); ++it) *it;
```

`vector`	accès O(1)	contigu, insertion O(n)
`list`	accès O(n)	chaînée, insertion O(1)
`map` `set`	O(log n)	arbre trié par clé
`unordered_map`	O(1) moy.	table de hachage

⚠ **Piège :** `push_back` peut réallouer : itérateurs, pointeurs et références vers un `vector` deviennent invalides.

Déplacement (C++11)




```
Buf(Buf &&o) noexcept       // déplacement : vole
    : data(o.data), n(o.n) { o.data = nullptr; }
Buf b = std::move(a);       // a reste valide, vide
```

`std::move` convertit seulement en `T&&`. **Règle des 5** : dtor, 2 copies, 2 déplacements ; **des 0** : n'en écrire aucun (`vector`, `unique_ptr`).

Exceptions

```
if (b == 0) throw std::invalid_argument("b nul");
try {
    r = safeDiv(a, b);
} catch (const std::invalid_argument &e) {
    std::cerr << e.what() << '\n';
} catch (const std::exception &e) {   // plus général
} catch (...) { throw; }              // tout : relance
```

`exception` ← `logic_error` (`invalid_argument`, `out_of_range`), `runtime_error`, `bad_alloc`. Locaux détruits à la remontée.